Architekturskizze: Flutter Cross-Platform Frontend für das SkyOS-Ökosystem

1. Systemübersicht & Rolle des Flutter-Frontends im SkyOS-Ökosystem

1.1 Systemkontext & Orchestrierung

Das **Flutter-Frontend** fungiert als zentrale, performante und plattformübergreifende Client-Anwendung (iOS, Android, Desktop, Web) zur Steuerung, Visualisierung und Orchestrierung des **SkyOS-Ökosystems**. Das unterliegende Backend-System basiert auf einer modularen, containerbasierten Multi-Dienst-Architektur, die über Docker Compose (`docker-compose.yml`, Specification Version 3.8) orchestriert wird.

Das Backend teilt sich in zwei primäre Container-Dienste auf:

- **`skyos_core_engine` (`skyos-orchestrator`):** Containerized auf Basis von `python:3.11-slim`. Führt die zentrale Steuerungslogik aus, beherbergt das **Gradio-Dashboard** (`dashboard_app.py`, Port 7860) sowie die **FastAPI-Kerninstanz** (`app.py`, Port 8000). Die Laufzeitumgebung bindet das Arbeitsverzeichnis `./:/workspace` dynamisch ein und verarbeitet globale System-Tokens.
- **`skyos_n8n_workflows` (`n8n-automation`):** Basiert auf dem offiziellen Image `docker.n8n.io/n8nio/n8n:latest` (Port 5678). Verwaltet automatisierte Workflows, Event-Trigger und Webhooks unter Absicherung durch HTTP-Basic-Auth (`N8N_BASIC_AUTH_ACTIVE=true`) und persistentem Docker-Volume `n8n_storage`.

Das Gesamtsystem erzwingt eine strikte **Zero-Cost-Infrastruktur-Philosophie**: Die Verarbeitungs- und Inferenzleistung wird ohne laufende Cloud-Hosting-Kosten über ein hybrides Modell bereitgestellt – bestehend aus **lokaler Inferenz** (z. B. Ausführung lokaler LLMs via LM Studio auf einer ThinkPad-Workstation) und kostenfreien **Free-Tier Cloud-Ressourcen** (Hugging Face Spaces SDK Deployments). Das Flutter-Frontend konsolidiert diese heterogenen Endpunkte in einer einheitlichen, reaktiven Benutzeroberfläche.

Infrastruktur-Hinweis zur Container-Topologie: Die globale `docker-compose.yml` deklariert innerhalb der Service-Definition `skyos_core_engine` die Umgebungsvariable `FASTAPI_PORT=8000`, schaltet in der Standard-Konfiguration jedoch lediglich den Port-Mapping-Eintrag `"7860:7860"` für die Gradio-Oberfläche frei. Um die direkte Erreichbarkeit der FastAPI-REST-Schnittstelle (`/` und `/health`) für exogene Flutter-Clients (insbesondere mobile Native-Builds und Web-Instanzen) zu gewährleisten, muss das

Port-Mapping in `docker-compose.yml` explizit um `"8000:8000"` erweitert oder über einen Reverse-Proxy (z. B. NGINX Ingress) geroutet werden.

1.2 Topologie der Dienst-Anbindung

Dienst / Komponente	Port / Endpunkt	Funktion & Frontend-Interaktion
FastAPI Core Engine	Port 8000 (`/`, `/health`)	Systemstatus-Exposition, Ausführung operativer Kerndienste, Microservice-Rest-Schnittstelle. Erfordert Freigabe von Port `8000:8000` im Container-Mapping.
Gradio Dashboard	Port 7860 (`skyos_core_engine`)	Orchestrierungsoberfläche für strategische Säulen (Pillar 3–5), Budget-Allokation (€2.5M Gesamtvolumen) und operative Master-Roadmap. Einbindung via Embedded WebViews / Dynamic Dashboards.
n8n Workflow Automation	Port 5678 (`skyos_n8n_workflows`)	Webhook-Triggering und Event-gesteuerte Prozess-Automatisierung. Authentifizierung via Basic Auth (`N8N_USER` / `N8N_PASSWORD`).
LM Studio Local	`http://localhost:1234/v1`	Lokale Open-Source-Modell-Inferenz über eine OpenAI-kompatible REST/Streaming API auf lokaler Hardware.

2. Frontend App-Architektur & Design Patterns

2.1 Clean Architecture & Schichtenmodell

Um maximale Wartbarkeit, Testbarkeit und Unabhängigkeit von externen Frameworks zu garantieren, folgt die Flutter-Anwendung strikt den Prinzipien der **Clean Architecture**. Das Projekt ist in drei isolierte Schichten gegliedert:

```
lib/
├── presentation/        # Presentation Layer (UI & State Management)
│   ├── bloc/            # BLoC- / Cubit-Implementierungen (z. B. SystemStatusBloc)
```

```
│   ├── pages/          # Responsive Screen layouts (Mobile, Tablet, Desktop)
│   └── widgets/        # Atomic UI-Komponenten
├── domain/             # Domain Layer (Pure Dart - Geschäftslogik)
│   ├── entities/       # Business Objects (z. B. PaperEntity, TelemetryReport)
│   ├── usecases/       # Pure Business Use Cases (z. B. SyncPapersUseCase)
│   └── repositories/   # Abstrakte Repository-Interfaces
└── data/               # Data Layer (Datenquellen & Infrastruktur)
    ├── datasources/    # Remote (Dio REST/WS) & Local (FlutterSecureStorage)
    ├── models/         # DTOs mit json_annotation / freezed Serialisierung
    ├── repositories/   # Konkrete Repository-Implementierungen
    └── interceptors/   # Security- & Auth-Http-Interceptors
```

1. **Presentation Layer (Präsentationsschicht)**
    - **Aufgaben:** Rendering plattformspezifischer UI-Widgets, Verwaltung von UI-States und Erfassung von Benutzerinteraktionen.
    - **Erlaubte Abhängigkeiten:** Greift ausschließlich auf die *Domain Layer* zu (Ausführung von Use Cases oder Injektion von BLoC-Zuständen via `flutter_bloc`). Keinerlei direkter Zugriff auf den Data Layer oder Netzwerktreiber.
2. **Domain Layer (Domänenschicht)**
    - **Aufgaben:** Enthält die Kern-Geschäftslogik, unmutable Entitäten (`Entities`), Use Cases und abstrakte Verträge (`Repository Interfaces`).
    - **Erlaubte Abhängigkeiten:** Absolut reine Dart-Bibliothek ohne Abhängigkeiten zu Flutter-UI oder konkreten Data-Layer-Paketen (*Dependency Inversion Principle*).
3. **Data Layer (Datenschicht)**
    - **Aufgaben:** Implementierung der Domain-Repository-Schnittstellen. Abwicklung von JSON-Deserialisierung (`json_serializable` / `freezed`), REST-/WebSocket-Kommunikation (`dio`), Zugriff auf lokale Tresore (`flutter_secure_storage`) sowie Caching.
    - **Erlaubte Abhängigkeiten:** Implementiert Interfaces der *Domain Layer* und interagiert direkt mit externen APIs, Datenbanken und Hardware-APIs.

2.2 State Management & Repository Pattern

Das State Management baut auf der Kombination von **BLoC (Business Logic Component)** und dem **Repository Pattern** auf.

- **Repository Pattern:** Kapselt Datenquellen (REST, WebSockets, Local Storage). Es entscheidet autonom über Strategien wie *Cache-First* oder *Network-First*. Zur funktionalen Fehlerbehandlung gibt das Repository Resultat-Typen im Format `Either<Failure, T>` (unter Nutzung von `fpdart`) an die Domänenschicht zurück, wodurch unkontrollierte Runtime-Exceptions verhindert werden.
- **BLoC Pattern (`flutter_bloc`):** Transformiert eingehende Event-Streams in unmutable State-Objekte (`StreamTransformer`). Zur Handhabung aufwendiger

JSON-Parsing-Aktionen (wie der Verarbeitung umfangreicher arXiv-Kataloge aus `papers/README.md`) lagert das Data Layer die Deserialisierung über Dart `Isolates` (`compute()`) aus, um Thread-Blocking auf der UI-Main-Isolate zu unterbinden.

3. Schnittstellen- & API-Integration

3.1 RESTful Endpoints & n8n Workflow-Triggering

Die Integration der Kern-APIs erfolgt über eine zentral konfigurierte `Dio`-Instanz:

- **FastAPI Kern-Schnittstellen:**
  - `GET /`: Validierung des Systemstatus.
    - *Expected Payload:* `{"status": "online", "system": "System-Health-Mindset Active"}`
  - `GET /health`: System-Gesundheitsprüfung.
    - *Expected Payload:* `{"health": "optimal"}` (HTTP 200 OK).
- **n8n Workflow-Triggering:**
  - **HTTP-Methode:** `POST` an n8n-Webhook-Endpunkte (`http://<host>:5678/webhook/...`).
  - **Payload-Format:** JSON-Payloads zur Übergabe von Parametern (z. B. Modul-Initialisierung, Location-Einträge wie `"Berlin"`).
  - **Fehlerbehandlung:** Evaluierung von HTTP-Statuscodes, konfigurierter Timeout (Default: 5000ms via `BaseOptions`) und automatische Fehler-Klassifizierung via Interceptor.

3.2 Echtzeit-Kommunikation & Telemetrie-Streaming

Zur Überwachung dynamischer System-Metriken baut das Frontend ein hybrides Streaming-Modell auf. Telemetriedaten werden wahlweise über WebSocket-Verbindungen oder hochfrequentes Polling verarbeitet. Eingehende Metriken befüllen automatisiert das Client-Seitige `Report_Template.md`:

SkyOS System Performance Report
- Zeitstempel: {{timestamp}}
- Auslastung ThinkPad / HF Spaces: {{metrics}}
- Fehlerrate Telemetrie: {{error_rate}}

Die Platzhalter `{{timestamp}}`, `{{metrics}}` und `{{error_rate}}` werden im Data-Layer über reaktive Stream-Transformationen gebunden und in Echtzeit an die Dashboard-Widgets emittiert.

3.3 Model Context Protocol (MCP) & Connector-Integration

Das Frontend integriert die im SkyOS-Ökosystem definierten MCP-Schnittstellen gemäß `Connectors.json`:

MCP-Konnektor	Typ / Endpunkt	Protokoll
`google-drive-mcp`	`type: remote`	`stdio`
`huggingface-space-mcp`	`type: sdk`	`fastmcp`
`lm-studio-local`	`endpoint: http://localhost:1234/v1`	`openai-compatible`

4. Sicherheits- & Token-Management (Platform Security)

4.1 Strikte Trennung von Code & Credentials

- **Absolutes Klartext-Verbot:** Es werden unter keinen Umständen Credentials, Tokens oder Private Keys im Quellcode, in Asset-Bundles oder Git-Repositories gespeichert.
- **Plattformnative Tresore:** Speicherung sensibler Secrets auf Client-Geräten erfolgt ausschließlich über `flutter_secure_storage` unter Ausnutzung plattformnativer Hardware-Sicherheitsmodule:
  - **iOS:** Keychain Services (mit `kSecAccessControlBiometryAny` Option).
  - **Android:** EncryptedSharedPreferences unterstützt durch den Android KeyStore System-Provider.
- **CI/CD & Git Security:** Secrets verbleiben in isolierten `.env`-Laufzeitumgebungen und werden per Pipeline injiziert.

4.2 Token-Lifecycle & Secret-Handling

Die Absicherung von Aufrufen gegen Backend-Dienste und externe APIs (`HF_TOKEN`, `GRAVATAR_API_KEY`, `GOOGLE_WORKSPACE_AUTH`, `N8N_USER`, `N8N_PASSWORD`) erfolgt dynamisch zur Laufzeit über einen dedizierten `Dio` Network Interceptor:

```
import 'dart:convert';
import 'package:dio/dio.dart';
import 'package:flutter_secure_storage/flutter_secure_storage.dart';

class SecurityAuthInterceptor extends Interceptor {
  final FlutterSecureStorage _secureStorage;

  SecurityAuthInterceptor(this._secureStorage);
```

```
  @override
  void onRequest(
    RequestOptions options,
    RequestInterceptorHandler handler,
  ) async {
    // Dynamisches Auslesen der verschlüsselten Secrets
    final hfToken = await _secureStorage.read(key: 'HF_TOKEN');
    final n8nUser = await _secureStorage.read(key: 'N8N_USER');
    final n8nPassword = await _secureStorage.read(key: 'N8N_PASSWORD');

    // Pfadbasierte Token-Injektion
    if (options.path.contains('n8n') && n8nUser != null && n8nPassword != null) {
      final basicAuth = 'Basic ${base64Encode(utf8.encode('$n8nUser:$n8nPassword'))}';
      options.headers['Authorization'] = basicAuth;
    } else if (hfToken != null) {
      options.headers['Authorization'] = 'Bearer $hfToken';
    }

    options.headers['Accept'] = 'application/json';
    handler.next(options);
  }
}
```

5. Integration von SkyAI Knowledge System & Telemetrie

5.1 Abbildung des Fünfstufigen Wissensmodells (Context Tiers L0–L4)

Das Data-Layer der Flutter-Anwendung bildet das fünfstufige Wissensmodell des SkyAI Knowledge Systems auf dedizierte Data Transfer Objects (DTOs) und Domain-Entitäten ab:

1. **`L0_Raw` (`RawApiDto`):** Unverarbeitete Rohdaten aus REST-Endpoints, System-Logs und Google Sheets.
2. **`L1_Extracted` (`ParsedJsonModel`):** Strukturierte JSON-Payloads und aufbereitete Vektor-Einbettungen.
3. **`L2_Processed` (`WorkflowTemplateModel`):** Synthetisierte Process-Flows und dynamische Prompt-Templates.
4. **`L3_Verified` (`VerifiedArtifactEntity`):** Auditierte, produktionsbereite Modul-Artefakte.
5. **`L4_Organized` (`SystemManifestModel`):** Globale Master Knowledge Base und `manifest.json` Instanzen.

Integration der Forschungsbibliothek (`sync_papers.py` Parsing): Das Skript `sync_papers.py` verarbeitet arXiv-IDs und speichert synchronisierte Arbeiten unter Einhaltung strikter Dateinamenskonventionen: Aus arXiv-IDs mit Punkten (z. B. `2310.07321`) werden Markdown-Dateien im Format `arxiv_2310_07321.md` im

Verzeichnis `papers/` generiert. Parallel aktualisiert das Skript die zentrale Indexdatei `papers/README.md`.

Der Flutter Data Layer parsed `papers/README.md` über einen dedizierten Parser-Dienst. Die Markdown-Tabelle wird extrahiert, in `PaperEntity`-Objekte konvertiert und im UI in einer filterbaren Data-Grid-Komponente visualisiert:

```
class PaperEntity {
  final String arxivId;      // z. B. "2310.07321"
  final String title;        // Paper-Titel
  final String publishedDate;// YYYY-MM-DD
  final String pdfUrl;       // Direct PDF Link
  final String localFileName;// z. B. "arxiv_2310_07321.md"

  const PaperEntity({
    required this.arxivId,
    required this.title,
    required this.publishedDate,
    required this.pdfUrl,
    required this.localFileName,
  });
}
```

5.2 ErrorTracker-Telemetrie & Offline-Pufferung

Fehlerereignisse innerhalb der Flutter-Laufzeitumgebung werden über eine native Implementierung der `ErrorTracker`-Spezifikation erfasst (äquivalent zu `errorTracking.js`). Der Logger zeichnet Exceptions auf, reichert sie mit Diagnose-Kontext an und übermittelt sie per HTTP `POST` an den Endpunkt `/api/telemetry`.

Bei Verbindungsunterbrechungen nutzt die Klasse das Paket `connectivity_plus` sowie ein lokales FIFO-Puffer-System (In-Memory / Hive Persistence), um Telemetriedaten bis zur Wiederherstellung der Netzwerkverbindung zwischenzuspeichern:

```
import 'dart:async';
import 'package:dio/dio.dart';
import 'package:connectivity_plus/connectivity_plus.dart';
import 'package:flutter/foundation.dart';

class ErrorTracker {
  final Dio _dio;
  final String endpoint;
  final List<Map<String, dynamic>> _offlineBuffer = [];
  bool _isProcessingBuffer = false;
```

```
ErrorTracker({
  Dio? dio,
  this.endpoint = "/api/telemetry",
}) : _dio = dio ?? Dio() {
  _initConnectivityListener();
}

void _initConnectivityListener() {
  Connectivity().onConnectivityChanged.listen((results) {
    if (!results.contains(ConnectivityResult.none) && _offlineBuffer.isNotEmpty) {
      _flushBuffer();
    }
  });
}

Future<void> captureException(
  Object error, {
  StackTrace? stackTrace,
  Map<String, dynamic>? context,
}) async {
  final payload = <String, dynamic>{:
    "timestamp": DateTime.now().toUtc().toIso8601String(),
    "name": error is Error ? error.runtimeType.toString() : "RuntimeError",
    "message": error.toString(),
    "stack": stackTrace?.toString(),
    "context": context ?? <String, dynamic>{},
  };

  debugPrint("[SkyOS Telemetry] Kritischer Fehler registriert: $payload");

  try {
    final connectivity = await Connectivity().checkConnectivity();
    if (connectivity.contains(ConnectivityResult.none)) {
      _bufferPayload(payload, "Keine Netzwerkverbindung.");
      return;
    }

    await _dio.post(
      endpoint,
      data: payload,
      options: Options(headers: {"Content-Type": "application/json"}),
    );
  } catch (e) {
    _bufferPayload(payload, e.toString());
  }
}

void _bufferPayload(Map<String, dynamic> payload, String reason) {
```

```
    _offlineBuffer.add(payload);
    debugPrint("[SkyOS Telemetry] Offline-Puffer aktiv: $reason (Gepuffert:
${_offlineBuffer.length})");
  }

  Future<void> _flushBuffer() async {
    if (_isProcessingBuffer || _offlineBuffer.isEmpty) return;
    _isProcessingBuffer = true;

    debugPrint("[SkyOS Telemetry] Leere Offline-Puffer (${_offlineBuffer.length}
Elemente)...");
    final itemsToSend = List<Map<String, dynamic>>.from(_offlineBuffer);

    for (final item in itemsToSend) {
      try {
        await _dio.post(
          endpoint,
          data: item,
          options: Options(headers: {"Content-Type": "application/json"}),
        );
        _offlineBuffer.remove(item);
      } catch (e) {
        debugPrint("[SkyOS Telemetry] Fehler beim Leeren des Puffers: $e");
        break; // Verbleibe im Puffer bei wiederholtem Fehler
      }
    }
    _isProcessingBuffer = false;
  }
}
```